

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a8d1d5200fa75b258.jsonl`  
- SHA-256: `31b254e7823730634dd5d702f1716673f2d5a075df463342503f4725f9c7d933`  
- Source modified: `2026-06-13T19:16:41+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:15:42.420Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"The tracknet substrate branch of _build_runner is never exercised end-to-end","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:58-68 ; tests/test_fusion_hybrid.py:47","detail": "_build_runner has a real branch on fusion.substrate ('tracknet' builds TrackNetRunner with queue_length + fusion_substrate='tracknet' params; default builds WasbRunner). Every fusion-hybrid test patches _build_runner out entirely (patch.object(FusionSegmenter, '_build_runner', return_value=(_runner(), {...}))), so neither branch runs and the runner_params dict ('fusion_substrate' tag, queue_length) is never asserted. test_config only checks that substrate='tracknet' parses into config, not that the segmenter routes on it. This is partially a GPU concern (constructing the real runners), but the routing logic + the returned params dict are pure and could be unit-tested by calling _build_runner directly with TrackNetConfig/WasbConfig mocked.", "fix": "Add a direct unit test of _build_runner: call seg._build_runner({'fusion':{'substrate':'tracknet'}}) and seg._build_runner({}) with TrackNetRunner/WasbRunner patched, asserting the right runner class is constructed and runner_params carries fusion_substrate plus weights_path (and queue_length for tracknet)."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:45.235Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:15:46.193Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).  
2  
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,  
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`  
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
```

```

6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9        GPU-free, computed from the trajectory we already have); and, only when the person cue
10       is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11       (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12       fusion (P3). The person path lazily builds ONE ``PersonDetector`` ? so the default path
13       never imports torch / touches the GPU.
14       - ``_select_for_handoff`` ? optional served Gate A/B: when
15       ``indexing.fusion.min_crossings``
16       (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17       are
18       dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19       **Persisted candidates remain the full superset** regardless, so the offline
20       experiment
21       harness can re-score any variant.
22
23       Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24       but
25       nothing is gated (thresholds 0, person OFF), so ``FusionSegmenter`` with default config
26       seeds
27       and hands off identically to ``wasb_hybrid``. Registered as ``fusion_hybrid``, NOT the
28       default
29       segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30       through the
31       experiment harness (EXPERIMENT_HARNESS.md), never silently.
32       """
33       from typing import Any, Dict, List, Optional, Tuple
34
35       from backend.pipeline.segmenters.base import segmenter_registry
36       from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37       from backend.pipeline.detectors.base import DetectorRunner
38       from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39       from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40       from backend.pipeline.detectors import rally_gate
41       from backend.pipeline.detectors import fusion_features as ff
42
43       class FusionSegmenter(TrajectoryHybridSegmenter):
44           """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45           B)."""
46
47           family = "fusion"
48           display_label = "Fusion"
49
50           def __init__(self, db: Any, config: Any):
51               super().__init__(db, config)
52               # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53               default path).
54               self._person: Optional[Any] = None
55
56           @property
57           def name(self) -> str:
58               return "fusion_hybrid"
59
60           @property
61           def description(self) -> str:
62               return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63               rallies, "
64
65                       "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
66               and the "
67
68                       "AI provider sets semantic boundaries.")
69
70           def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
71               Any]]:

```

```

59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                frame_width: float, video_path: str,
72                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                          frame_width: float, video_path: str,
87                          fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                             window_stats: List[Dict[str, Any]],
111                             idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds

```

```

113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:15:46.353Z)

```

1     """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3     Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4     keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5     substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6     persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7     """
8     from types import SimpleNamespace
9     from unittest.mock import MagicMock, patch
10
11     from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14     # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17             "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19             "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22     def _points():
23         pts = []
24         for f in range(30, 120):             # window A frames: y flips 100<->900 around the net
(~500)
25             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26         for f in range(300, 360):             # window B frames: y == net level -> deadband -> no
crossings
27             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28         return pts
29
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]

```

```

37         return r
38
39
40     def _run(indexing=None, provider_segments=None):
41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43         db = MagicMock()
44         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45         db.add_play_segment.return_value = 200
46
47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
49             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52             patch("os.environ.get", return_value="dummy_key"):
53             preg.get.return_value = provider
54             seg = FusionSegmenter(db, {"indexing": indexing or {}})
55             res = seg.process_video("v1", "clip.mp4")
56             return db, provider, res
57
58
59     def _stats_calls(db):
60         """Persisted stats dicts, in call order."""
61         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64     def test_net_crossings_persisted_and_person_off_by_default():
65         db, _provider, _res = _run()
66         stats = _stats_calls(db)
67         assert len(stats) == 2                                # both windows persisted
(superset)
68         assert all("net_crossings" in s for s in stats)
69         # Window A (rally) crosses many times; window B (held) zero.
70         assert stats[0]["net_crossings"] >= 2
71         assert stats[1]["net_crossings"] == 0.0
72         # Person cue OFF by default -> NONE of the person features present.
73         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
74         # The 4 base motion keys are still there.
75         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76                     for s in stats)
77
78
79     def test_persisted_candidates_are_superset_of_substrate_windows():
80         db, _provider, _res = _run()
81         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82         assert spans == [(1.0, 4.0), (10.0, 12.0)]            # exactly the substrate windows,
unfiltered
83
84
85     def test_default_hands_off_all_windows():
86         _db, provider, _res = _run()                            # min_crossings default 0 -> no
gating
87         assert provider.analyze_video.call_count == 2
88
89
90     def test_gate_a_drops_low_crossing_window_from_handoff_only():

```

```

91         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92         # Persistence is still the full superset (offline substrate intact)...
93         assert len(_stats_calls(db)) == 2
94         # ...but only the high-crossing window A reaches the AI handoff.
95         assert provider.analyze_video.call_count == 1
96
97
98     def test_person_cue_adds_features_when_enabled():
99         fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
105             return_value=fake_pd):
106             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
107             stats = _stats_calls(db)
108             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
109             assert all("net_crossings" in s for s in stats)
110             fake_pd.detections_per_frame.assert_called() # the person detector WAS used

```

5. assistant (2026-06-13T19:15:50.143Z)

Now let me verify the claims in the finding. Let me check the parent class to understand `\_build\_runner`, `from\_indexing\_cfg` signatures, and look at the config test mentioned.

6. user (2026-06-13T19:15:51.121Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36

```

```

37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47         #: config section under `indexing` (velocity_threshold lives there), the
48         #: output subdir for trajectories, and the metadata detector-tag prefix.
49         family: str = ""
50         #: human-readable label used in log lines.
51         display_label: str = ""
52
53     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54         """Return (runner, detector-specific detection_params extras)."""
55         raise NotImplementedError
56
57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
87         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
88         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
89         features). ``points`` are the whole-video trajectory points
90         (TrajectoryPoint)."""
91         return {
92             "peak_velocity": cw["peak_velocity"],
93             "mean_velocity": cw["mean_velocity"],
94             "active_frame_count": cw["active_frame_count"],
95             "track_id_count": cw["track_id_count"],
96         }

```

```

96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                             window_stats: List[Dict[str, Any]],
98                             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                      player_pool: Optional[List[str]] = None,
108                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156

```



```

157     detection_params = {
158         "detector": self.name,
159         "velocity_thresh": velocity_thresh,
160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184                 len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                 label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label}] rally]
197             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198                 f"({cw['end'] - cw['start']:.1f}s) |
199             frames={cw['active_frame_count']} "
200                 f"peak_v={cw['peak_velocity']:.3f}
201             mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     fusion
205     # segmenter adds net_crossings + person-cue features. Computed once, reused for
206     both
207     # persistence and the handoff gate below.
208     window_stats = [
209         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210         idx_cfg)
211         for cw in all_candidate_windows
212     ]
213
214     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
215     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
216     if store_candidates:
217         for i, cw in enumerate(all_candidate_windows):
218             candidate_ids[i] = self.db.add_candidate_segment(
219                 video_id, detector=self.name,
220                 start_time=cw["start"], end_time=cw["end"],
221                 chunk_index=cw["chunk_index"], confidence=min(1.0,

```

```

cw["peak_velocity"]],
216         stats=window_stats[i], detection_params=detection_params,
217     )
218
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224     # candidates above stay the full superset ? only the served play_segments are
gated.
225     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227     os.makedirs(handoff_dir, exist_ok=True)
228     logger.info("Phase 3: %s validation on %d candidate windows...",
229                 provider_name, len(handoff_indices))
230
231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                             label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["_candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                     for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262     try:
263         os.rmdir(handoff_dir)
264     except OSError:
265         pass
266     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:

```

```

272         start = float(segment["start_time"])
273         end = float(segment["end_time"])
274         dur = end - start
275         metadata = {
276             "shot_count": self._shot_count_from(segment, dur),
277             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278             "detector": f"{self.family}-hybrid-{model_name}",
279         }
280         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281         if segment_id:
282             candidate_id = segment.get("_candidate_id")
283             if candidate_id is not None:
284                 self.db.link_candidate_to_segment(candidate_id, segment_id)
285             segments_data.append({
286                 "id": segment_id, "start_time": start, "end_time": end,
287                 "duration": dur, "metadata": metadata,
288             })
289     except Exception as e:
290         logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

7. user (2026-06-13T19:15:52.208Z)

docs\CODE_MAP.md:50:### 1B. WASB shuttle substrate (P2 of wasb_hybrid)

docs\CODE_MAP.md:114:- `@backend/pipeline/segmenters/wasb\_hybrid.py::WasbHybridSegmenter` (name `wasb\_hybrid`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection\_params` extras = `weights\_path` only.

docs\CODE_MAP.md:140:- `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO\_FEATURE\_NAMES` (5-d), `::extract\_yolo\_features` (? reads `row['stats']` dict from DB; missing fields default 0.0), `::label\_candidates` (same IoU `match\_intervals` as evaluator), `::build\_training\_set`->(X,y,intervals); `feature\_fn` is the substrate plug-in seam.

backend\config\models.py:55: """Typed config for the live WASB shuttle substrate (indexing.wasb).

backend\config\models.py:88: # Which trajectory substrate seeds the windows (shuttle stays primary).

backend\config\models.py:89: substrate: Literal["wasb", "tracknet"] = "wasb"

backend\config\models.py:91: # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set

docs\COMMERCIALIZATION.md:85: cost, no external dep) and matches the Phase-4 roadmap (substrate = WASB

backend\reporting\spec.yaml:94: # for good results with the shuttle-tracking substrates.

backend\pipeline\segmenters\wasb_hybrid.py:5:badminton weights and runs today, so this is the live shuttle-motion substrate.

tests\test_config.py:29: """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no mask."""

tests\test_config.py:32: assert f.substrate == "wasb"

tests\test_config.py:45: "substrate": "tracknet", "min_crossings": 2,

tests\test_config.py:50: assert f.substrate == "tracknet"

tests\test_config_wasb.py:5:shuttle substrate's config) was discarded ? config.json overrides did nothing.

docs\EXPERIMENT_HARNESS.md:50:stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence substrate.

docs\EXPERIMENT_HARNESS.md:88:### 3.5 Offline re-scoring substrate (persist once, re-score forever)

docs\archives\COMMERCIAL_READINESS_REVIEW.md:71: - Validate real-video performance of the current substrates (WASB is live default).

backend\pipeline\segmenters\trajectory_hybrid.py:1:"""Shared engine for trajectory-substrate hybrid segmenters.

docs\archives\decisions\DECISIONS.md:153:**Status:** Proposed (substrate pending data)

docs\archives\decisions\DECISIONS.md:167:Three candidate substrates were weighed:

docs\archives\decisions\DECISIONS.md:183:2. ****Choose the substrate from data, not assumption.****
A learned refiner can only sharpen
docs\archives\decisions\DECISIONS.md:199:### Update (2026-05-26): Phase 3 evidence ? substrate is WASB, not YOLO
docs\archives\decisions\DECISIONS.md:218:shuttle signal), the offline segmenter's substrate is ****WASB shuttle-motion**** (ADR-001),
docs\archives\decisions\DECISIONS.md:227:substrate-agnostic training-data + classifier layers already built still apply: they will
docs\archives\decisions\DECISIONS.md:302:After building the WASB shuttle-trajectory substrate (ADR-001), a Gemini-teacher ? local-student distillation
docs\GOLDEN_SET_PHASE1_VIDEOS.md:40:A controlled study: table-tennis ball detection was ****22.5% @ 30 fps ? 93.2% @ 60 fps**** (120 fps ? 60 fps) ([Nature Sci Rep](https://www.nature.com/articles/s41598-023-42966-6)); OpenTTGames records at 120 fps. The ball/shuttle is the camera-invariant rally signal (our WASB substrate, ADR-001), so fps is ****not**** a nicety for fast sports.
backend\eval\experiment.py:6:substrate) already exists ? this module composes it. ****No new scoring math lives**
backend\pipeline\segmenters\fusion_hybrid.py:3:Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
backend\pipeline\segmenters\fusion_hybrid.py:20:Defaults reproduce the substrate exactly: net_crossings/person features are persisted but
backend\pipeline\segmenters\fusion_hybrid.py:59: substrate = idx_cfg.get("fusion", {})).get("substrate", "wasb")
backend\pipeline\segmenters\fusion_hybrid.py:60: if substrate == "tracknet":
backend\pipeline\segmenters\fusion_hybrid.py:65: "fusion_substrate": "tracknet",
backend\pipeline\segmenters\fusion_hybrid.py:68: return WasbRunner(wcfg),
{"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}
backend\pipeline\segmenters\fusion_hybrid.py:113: (0) keep every window ? identical to the substrate. Persisted candidates are
backend\eval\training.py:15: persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
backend\eval\training.py:19:substrate is chosen from the Phase 3 candidate-recall result.
backend\eval\training.py:72: `feature\_fn` is pluggable so a WASB substrate can supply shuttle-motion
backend\pipeline\detectors\base.py:5:the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
tests\test_fusion_hybrid.py:5:substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
tests\test_fusion_hybrid.py:79:def
test_persisted_candidates_are_superset_of_substrate_windows():
tests\test_fusion_hybrid.py:82: assert spans == [(1.0, 4.0), (10.0, 12.0)] # exactly
the substrate windows, unfiltered
tests\test_fusion_hybrid.py:92: # Persistence is still the full superset (offline substrate intact)...
docs\ROADMAP.md:71:payoff ? a free, offline, self-improving rally mode. The ****substrate was chosen from**
docs\ROADMAP.md:75:substrate is ****WASB shuttle-motion**** (ADR-001), not player-motion. See ****ADR-004****.
docs\ROADMAP.md:79:Once the substrate is WASB shuttle-motion, the next lever is ****fusing additional**
docs\ROADMAP.md:114:| 4 | Offline learned segmenter (WASB shuttle-motion substrate) | ? Next (substrate decided: WASB, ADR-004) |
docs\VIDEO_RECORDING_GUIDELINES.md:16:[Omitted long matching line]

8. assistant (2026-06-13T19:15:55.127Z)

Now let me verify the test_config.py claim about whether it checks routing or just config parsing.

9. user (2026-06-13T19:15:56.130Z)

```
25         assert cfg.indexing.ai_max_workers == 5
26
27
28     def test_fusion_config_defaults_reproduce_control():
```

```

29         """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
mask."""
30         cfg = load_config("/nonexistent/path/12345/config.json")
31         f = cfg.indexing.fusion
32         assert f.substrate == "wasb"
33         assert f.cue_flags == {} # person cue OFF
34         assert f.min_crossings == 0 # Gate A OFF
35         assert f.min_players == 0 # Gate B OFF
36         assert f.court_polygon is None # no mask
37         assert f.velocity_threshold == 0.02
38
39
40     def test_fusion_config_file_override():
41         with tempfile.TemporaryDirectory() as td:
42             p = os.path.join(td, "fusion.json")
43             with open(p, "w") as fh:
44                 json.dump({"indexing": {"fusion": {
45                     "substrate": "tracknet", "min_crossings": 2,
46                     "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1],
[0.5, 0.9]],
47                 }}, fh)
48             cfg = load_config(p)
49             f = cfg.indexing.fusion
50             assert f.substrate == "tracknet"
51             assert f.min_crossings == 2
52             assert f.cue_flags == {"person": True}
53             assert len(f.court_polygon) == 3
54             # An old config without a fusion block still loads (defaults applied).
55             assert cfg.indexing.wasb.velocity_threshold == 0.02
56
57
58     def test_file_override():
59         with tempfile.TemporaryDirectory() as td:

```

10. user (2026-06-13T19:15:56.910Z)

```

docs\CODE_MAP.md:113:[Omitted long matching line]
backend\pipeline\segmenters\tracknet_hybrid.py:34:     def _build_runner(self, idx_cfg: Dict[str,
Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\wasb_hybrid.py:36:     def _build_runner(self, idx_cfg: Dict[str,
Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\fusion_hybrid.py:58:     def _build_runner(self, idx_cfg: Dict[str,
Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\trajectory_hybrid.py:15:- ``_build_runner(idx_cfg)`` ? returns
(DetectorRunner, detector-specific
backend\pipeline\segmenters\trajectory_hybrid.py:53:     def _build_runner(self, idx_cfg:
Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\trajectory_hybrid.py:147:         runner, runner_params =
self._build_runner(idx_cfg)
tests\test_fusion_hybrid.py:47:     with patch.object(FusionSegmenter, "_build_runner",
return_value=(_runner(), {"weights_path": "w"})), \
tests\test_trajectory_hybrid_equivalence.py:48:     with patch.object(cls, "_build_runner",
return_value=(_runner(), {"weights_path": "w"})), \

```

11. assistant (2026-06-13T19:16:02.061Z)

Now let me verify whether the `TrackNetConfig.from_indexing_cfg`` and `WasbConfig.from_indexing_cfg`` accept `idx_cfg`` and that the `queue_length`` attribute exists, so that the suggested direct unit test would actually be feasible/correct.

12. user (2026-06-13T19:16:03.343Z)

```

backend\pipeline\detectors\tracknet_runner.py:37:     weights_path: str = ""
# WSL path to the .h5/.keras weights (REQUIRED)

```

```

backend\pipeline\detectors\tracknet_runner.py:38: queue_length: int = 5
backend\pipeline\detectors\tracknet_runner.py:46: def from_indexing_cfg(cls, idx_cfg:
Dict[str, Any]) -> "TrackNetConfig":
backend\pipeline\detectors\tracknet_runner.py:53: weights_path=tn.get("weights_path",
""),
backend\pipeline\detectors\tracknet_runner.py:54:
queue_length=int(tn.get("queue_length", 5)),
backend\pipeline\detectors\tracknet_runner.py:130: if not self.cfg.weights_path:
backend\pipeline\detectors\tracknet_runner.py:132:     "TrackNet weights_path is not
set. Set indexing.tracknet.weights_path "
backend\pipeline\detectors\tracknet_runner.py:184: f"--model_weights
{wsl_tilde_quote(self.cfg.weights_path)} "
backend\pipeline\detectors\tracknet_runner.py:186: f"--queue_length
{self.cfg.queue_length}"

```

13. user (2026-06-13T19:16:03.948Z)

```

backend\pipeline\detectors\wasb_runner.py:37: weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
backend\pipeline\detectors\wasb_runner.py:45: def from_indexing_cfg(cls, idx_cfg: Dict[str,
Any]) -> "WasbConfig":
backend\pipeline\detectors\wasb_runner.py:52: weights_path=w.get("weights_path",
backend\pipeline\detectors\wasb_runner.py:149: f"--weights
{wsl_tilde_quote(self.cfg.weights_path)} "

```

14. assistant (2026-06-13T19:16:22.015Z)

I now have all the facts needed to adjudicate. Let me verify each claim in the finding against the code.

****Claim 1: `\_build\_runner` has a real branch on `fusion.substrate`.** Confirmed ?**
`fusion\_hybrid.py:58-68`. Line 59 reads `substrate = idx\_cfg.get("fusion", {}).get("substrate", "wasb")`; line 60 branches; tracknet builds `TrackNetRunner(cfg)` with params `{weights\_path, queue\_length, fusion\_substrate: "tracknet"}`; the default builds `WasbRunner(wcfg)` with `{weights\_path, fusion\_substrate: "wasb"}`.

****Claim 2: Every fusion-hybrid test patches `\_build\_runner` out entirely.** Confirmed ?**
`test\_fusion\_hybrid.py:47`: `patch.object(FusionSegmenter, "\_build\_runner", return\_value=(\_runner(), {"weights\_path": "w"}))`. All five tests route through `\_run()`, which uses this patch. So neither branch of the real `\_build\_runner` runs, and the returned `runner\_params` (with `fusion\_substrate` / `queue\_length`) is never asserted. The mock return even omits `fusion\_substrate`, confirming it isn't exercised.

****Claim 3: `test\_config` only checks parsing, not routing.** Confirmed ?** `test\_config.py:40-55` (`test\_fusion\_config\_file\_override`) asserts `f.substrate == "tracknet"` on the parsed config object only; it never instantiates `FusionSegmenter` or calls `\_build\_runner`. So nothing asserts the segmenter routes on the substrate value.

****Claim 4: The routing + returned params dict are pure and unit-testable directly.** Confirmed feasible ?** `\_build\_runner` only constructs config via `TrackNetConfig.from\_indexing\_cfg(idx\_cfg)` / `WasbConfig.from\_indexing\_cfg(idx\_cfg)` (both pure dict readers, `tracknet\_runner.py:46`, `wasb\_runner.py:45`) and constructs the runner. With `TrackNetRunner` / `WasbRunner` patched, calling `\_build\_runner` directly is pure logic. `cfg.queue\_length` (`tracknet\_runner.py:38`) and `cfg.weights\_path` (`tracknet\_runner.py:37`, `wasb\_runner.py:37`) exist, so the suggested assertions are valid. The fix is accurate and achievable in this no-GPU environment.

****Severity check:**** This is a genuine coverage gap ? the substrate-routing branch and the `fusion\_substrate` / `queue\_length` metadata tagging are entirely untested, and that metadata flows into `detection\_params` persisted to the DB (`trajectory\_hybrid.py:162`). But it's a low-severity test-coverage observation, not a correctness bug. The code itself is correct; only the test suite under-covers it. The finding self-rates "low" and explicitly frames it as a test gap, not a nitpick about style. That matches.

Everything in the finding holds against the actual code. No part is refuted.

15. user (2026-06-13T19:16:34.186Z)

Structured output provided successfully

16. assistant (2026-06-13T19:16:41.433Z)

I've verified the finding against the actual code and confirmed it holds (real=true, severity=low).

Summary of what I checked:

- ``backend/pipeline/segmenters/fusion_hybrid.py:58-68`` ? the ``substrate`` branch is real and returns distinct ``runner_params`` dicts (``fusion_substrate`` tag, plus ``queue_length`` for tracknet).
- ``tests/test_fusion_hybrid.py:47`` ? all five tests route through one ``_run()`` helper that patches ``_build_runner`` out entirely, returning ``{"weights_path": "w"}`` (notably omitting ``fusion_substrate``), so neither branch ever runs.
- ``tests/test_config.py:40-55`` ? only asserts the parsed config object (``f.substrate == "tracknet"``), never instantiates the segmenter or calls ``_build_runner``, so routing is untested.
- ``backend/pipeline/segmenters/trajectory_hybrid.py:162,216`` ? confirmed ``runner_params`` flows into persisted ``detection_params``, so the untested tag isn't inert.
- ``tracknet_runner.py:37-38,46`` and ``wasb_runner.py:37,45`` ? confirmed ``from_indexing_cfg``, ``queue_length``, and ``weights_path`` exist, so the suggested direct unit test is feasible and offline-runnable.

It's a genuine test-coverage gap (the substrate-routing branch and its metadata are entirely untested), correctly self-rated low since the production code itself is correct.